UME

Organisation / Business Operating System

Low-Level Software Design Specification

Document ID: UME-LLD-001 | Version: 2.0.0

Status: Updated — OrgOS Factory Integration | Date: March 2026

Language: Rust (platform) · Java (client suite)

Coverage: OrgOS · OrgKernel · OrgApps · ODD Subsystem · SDO Engine · DAO/DLT Module · OrgDNA

Document Change History & Coverage

Version	Date	Summary
1.0.0	Jan 2026	Baseline LLD: Kernel · 42 modules · device drivers · API · data · security · deployment
2.0.0	Mar 2026	OrgOS runtime design · OrgApp trait · ODD subsystem · SDO/Digital Twin engine · DAO & DLT module · OrgDNA serialisation · OrgExec/OrgCPU registry

New Coverage in v2.0.0

Section	Content
§2 OrgOS Runtime	OrgOS struct, OrgApp trait (extends KernelModule), OrgDNA loader, boot sequence extensions
§3 ODD Subsystem	OrgDeviceDriver trait, ODDBus, OrgExecRegistry, OrgCPURegistry, ODD enforcement engine, violation logging
§4 SDO Engine	SDOEngine struct, topology graph (petgraph), state aggregation, OrgSimulator interface
§5 OrgDNA System	OrgDNA struct, serialisation (serde + TOML/JSON), version management, export/import API, diff engine
§6 DAO & DLT Module	DAOClient, DLTClient, consensus integration, smart contract compiler + executor, inter-org event bus
§7 Updated Module Designs	OrgApp extensions for v2.0.0 across Finance, HR, Risk, Process, and new OrgExec/ODD Manager app
Appendix E	Updated event catalogue including ODD, SDO, DAO, and DLT event topics
Appendix F	Updated glossary with all v2.0.0 terms

1. System Overview & Design Philosophy (Updated)

v2.0.0: OrgOS Factory — one platform, infinite organisation operating systems

All eight architecture principles from v1.0.0 remain in force (Kernel Mediation, Domain Isolation, Event-Driven Integration, RBAC-First Security, Fault Isolation, Immutable Audit, Composable Modules, Schema-First Data). v2.0.0 adds four new principles:

New Principle (v2.0.0)	Statement	Consequence
Uniform Actor Governance	All actors — humans, AI agents, machines, IoT devices — are governed by ODD contracts enforced at the OrgKernel level	No special-case actor handling; AI agents subject to same enforcement as human users
Digital Twin Completeness	Every OrgApp event updates the SDO in real time; the digital twin is never stale by more than one event	Single source of truth for org state; exec dashboards reflect reality within event propagation time
OrgDNA as Configuration Truth	The OrgDNA package is the sole authoritative source for OrgOS configuration; no config drift possible	All config changes produce a new OrgDNA version; full config history maintained
DAO-Native Inter-Org	Any two or more OrgOS instances can form a DAO without platform migration; the DLT layer is a first-class concern	Horizontal/vertical scaling via DAO requires zero re-architecture

1.1 Updated Key Design Decisions

Decision	Rationale	Implementation Note
ODD enforcement in ODDBus	All actor action validation centralised in kernel subsystem; no per-OrgApp enforcement logic	ODDBus.validate_action() called by kernel facade before dispatching any OrgExec-originated call
SDO as event-sourced read model	Digital twin built from OrgApp events; no separate write path; always consistent with transactional data	SDOEngine subscribes to all domain event topics; maintains in-memory topology + state graph
OrgDNA as TOML + versioned blobs	Human-readable for debugging; efficient for storage and diff; semantic versioning enforced	serde_derive with TOML feature; diff computed with diffy crate; stored in Postgres JSONB + S3
DLT as permissioned ledger	DAO members are known, authenticated OrgOS nodes; permissioned consensus is faster and more appropriate than public chain	Default: PBFT consensus among DAO members; optional: anchor to public chain for notarisation
Smart contracts as Rust closures	Compile-time verified; no separate VM overhead; fully integrated with OrgKernel type system	Smart contract terms are Rust closures stored as serialised bytecode; executed by SmartContractEngine

2. OrgOS Runtime Design (New in v2.0.0)

OrgOS Runtime Design

Each OrgOS instance is an independent runtime — a single process containing an OrgKernel and all its active OrgApps, OrgExecs, and connected OrgCPUs. The OrgOS runtime is spawned by the UME meta-kernel (the factory) and operates autonomously once provisioned.

OrgOS Struct

OrgOS Runtime (ume_core::orgsys)

```
pub struct OrgOS {
    org_id:       OrgId,
    org_dna:      OrgDNA,          // versioned master config
    kernel:       OrgKernel,       // composition root
    app_registry: OrgAppRegistry, // all active OrgApps
    exec_registry:OrgExecRegistry,// all OrgExecs + their ODDs
    cpu_registry: OrgCPURegistry, // all OrgCPUs + their ODDs
    sdo:          SDOEngine,       // digital twin state engine
    dao_client:   Option<DAOClient>, // present if DAO member
    dlt_client:   Option<DLTClient>, // present if DAO member
}
```

OrgApp Trait (replacing KernelModule)

Organisation modules are now formally termed OrgApps. The OrgApp trait extends the previous KernelModule trait with OrgOS-specific metadata for marketplace display, configuration UI, dependency management, and OrgDNA serialization.

OrgApp Trait (ume_core::orgapp)

```
pub trait OrgApp: KernelModule {
    fn app_id(&self) -> &'static str;         // marketplace unique ID
    fn app_version(&self) -> SemanticVersion;
    fn app_category(&self) -> AppCategory;    // Finance | HR | Legal | ...
    fn app_tier(&self) -> AppTier;            // Core | Starter | Growth | Enterprise
    fn config_schema(&self) -> JsonSchema;    // drives UI config wizard
    fn to_orgdna_component(&self) -> OrgDNAComponent; // serialise to OrgDNA
    fn from_orgdna_component(c: OrgDNAComponent) -> UmeResult<Self>; // restore from OrgDNA
    fn marketplace_metadata(&self) -> AppMarketplaceEntry;
}
```

2.3 OrgOS Provisioning Sequence

The UME factory (meta-kernel) provisions a new OrgOS instance as follows:

Step	Operation	Rust API
1	Validate OrgDNA template or fresh config	OrgDNALoader::validate(dna: &OrgDNA) -> UmeResult<()>

2	Allocate infrastructure: database schema namespace, S3 bucket prefix, executor threads	OrgOSFactory::allocate_resources(org_id, dna) -> ResourceHandle
3	Run database migrations for all enabled OrgApps	StorageManager::migrate_all(org_id, app_list)
4	Instantiate OrgKernel with OrgDNA config	OrgKernel::new(org_id, dna, resources) -> UmeResult<OrgKernel>
5	Register all ODD templates from OrgDNA	ODDBus::load_from_orgdna(dna.odd_library)
6	Boot all enabled OrgApps in dependency order	OrgAppRegistry::boot_all(kernel) -> BootReport
7	Initialize SDO engine with OrgApp state snapshots	SDOEngine::initialize(kernel) -> SDOSnapshot
8	Connect DAO/DLT clients if org is DAO member	DAOClient::connect(dna.dao_config) -> Option<DAOHandle>
9	Emit OrgOS.provisioned event; expose HTTP endpoints	kernel.event_bus.publish(OrgOSProvisionedEvent)

2.4 OrgApp Trait (Full Specification)

OrgApp Trait — Full Definition

```
/// Extends KernelModule with OrgOS-specific capabilities
pub trait OrgApp: KernelModule + Send + Sync {
    // Marketplace identity
    fn app_id(&self) -> &'static str;
    fn app_version(&self) -> SemanticVersion;
    fn app_category(&self) -> AppCategory;
    fn app_tier(&self) -> AppTier;

    // OrgDNA integration
    fn to_orgdna_component(&self) -> OrgDNAComponent;
    fn from_orgdna_component(c: OrgDNAComponent) -> UmeResult<Box<dyn OrgApp>>;
    fn config_schema(&self) -> JsonSchema;  // drives setup wizard UI

    // Marketplace
    fn marketplace_metadata(&self) -> AppMarketplaceEntry;

    // SDO integration (default impl: no-op)
    fn sdo_state(&self) -> SDOAppState { SDOAppState::default() }
    fn sdo_topology_nodes(&self) -> Vec<SDONode> { vec![] }

    // ODD integration
    fn odd_capability_grants(&self) -> Vec<CapabilityGrant>;
    fn validate_odd_action(&self, action: &OrgAction, odd: &dyn OrgDeviceDriver) -> OddDecision;
}
```

3. ODD Subsystem Design (New in v2.0.0)

The ODD (OrgDeviceDriver) subsystem is the kernel component that manages all OrgExec and OrgCPU actor contracts. It is the enforcement engine for the uniform actor governance model.

3.1 OrgDeviceDriver Trait

OrgDeviceDriver Trait (ume_core::odd)

```
pub trait OrgDeviceDriver: Send + Sync {
    fn odd_id(&self) -> OddId;
    fn odd_version(&self) -> SemanticVersion;
    fn exec_type(&self) -> OrgExecType;
    fn capabilities(&self) -> &[OddCapability];
    fn constraints(&self) -> &[OddConstraint];
    fn data_access_grants(&self) -> &[DataAccessGrant];
    fn escalation_rules(&self) -> &[EscalationRule];
    fn audit_config(&self) -> &AuditConfig;
    fn lifecycle_policy(&self) -> &LifecyclePolicy;

    /// Validate a single action against this ODD
    fn validate_action(&self, action: &OrgAction, ctx: &OrgContext) -> OddDecision;

    /// Called by ODDBus on each tick; checks health of this actor
    fn health_check(&self) -> OddHealthReport;
}
```

3.2 ODDBus

ODDBus struct (ume_kernel::odd_bus)

```
pub struct ODDBus {
    contracts: HashMap<OrgExecId, Arc<dyn OrgDeviceDriver>>,
    violation_log: ViolationLog,
    escalation_router: EscalationRouter,
    audit: Arc<LogAuditManager>,
}

impl ODDBus {
    pub fn register(&mut self, exec_id: OrgExecId, odd: impl OrgDeviceDriver + 'static)
        -> UmeResult<()>;

    pub fn validate_action(&self, exec_id: &OrgExecId, action: &OrgAction)
        -> OddDecision;  // Allow | Deny(reason) | Escalate(rule)

    pub fn update_contract(&mut self, exec_id: &OrgExecId, new_odd: impl OrgDeviceDriver)
        -> UmeResult<()>;

    pub fn list_violations(&self, exec_id: &OrgExecId, window: TimeWindow)
        -> Vec<OddViolation>;
```

```
    pub fn health_check_all(&self) -> Vec<OddHealthReport>;
}
```

3.3 ODD Enforcement Flow

Step	Component	Operation
1	HTTP/CLI Layer	Request arrives with actor credentials (JWT or ODD identity token)
2	AuthMiddleware	Authenticate actor identity; attach OrgExecId to request context
3	KernelFacade	Before dispatching operation, call ODDBus::validate_action(exec_id, action)
4	ODDBus	Load actor ODD; evaluate all capabilities, constraints, and data access rules
5a	OddDecision::Allow	Action proceeds; ODD audit log entry written
5b	OddDecision::Deny(reason)	Action rejected; 403 returned; ODD violation logged; RBAC denial audit recorded
5c	OddDecision::Escalate(rule)	Action paused; escalation event published; assigned human OrgExec notified for approval
6	OrgApp	If allowed: executes operation; emits domain event; updates SDO
7	SDOEngine	Receives domain event; updates digital twin state

3.4 ODD Data Structures

Type	Fields	Notes
OddCapability	{action: String, targets: Vec<ResourceTarget>, conditions: Vec<Condition>}	e.g. Write:Ops.orders where assigned_to == self
OddConstraint	{constraint_type: ConstraintType, params: JsonValue, on_violation: ViolationAction}	RateLimit | TimeWindow | ApprovalThreshold | DataScope
DataAccessGrant	{app_id: String, resource: String, fields: FieldSelector, access: AccessLevel}	Read | Write | Admin; field-level granularity
EscalationRule	{trigger: EscalationTrigger, escalate_to: OrgExecId, timeout_mins: u32, fallback: FallbackAction}	Trigger: ConflictDetected | ConfidenceBelowThreshold | ErrorRateExceeded
OddDecision	Allow | Deny{reason: String} | Escalate{rule: EscalationRule}	Returned by validate_action()
OddViolation	{id, exec_id, timestamp, action, constraint_id, resolution}	Append-only log; searchable by exec, time, constraint

4. SDO Engine Design (New in v2.0.0)

The SDOEngine is the kernel subsystem responsible for maintaining the Software Defined Organisation: the live digital twin of the organisation.

4.1 SDOEngine Struct

SDOEngine (ume_kernel::sdo)

```
pub struct SDOEngine {
    org_id:      OrgId,
    topology:    Arc<RwLock<OrgTopologyGraph>>,  // petgraph::DiGraph
    state:       Arc<RwLock<OrgStateMap>>,       // HashMap<AppId, AppState>
    history:     Arc<AppendLog<SDOEvent>>,
    simulator:   Option<OrgSimulator>,
    optimiser:   Option<OrgOptimiser>,
}

impl SDOEngine {
    pub fn on_event(&mut self, event: KernelEvent) -> UmeResult<()>;
    pub fn get_topology(&self) -> OrgTopologySnapshot;
    pub fn get_org_state(&self) -> OrgStateSnapshot;
    pub fn run_simulation(&self, scenario: SimScenario) -> SimResult;
    pub fn get_optimisation_suggestions(&self) -> Vec<OrgSuggestion>;
    pub fn export_snapshot(&self) -> SDOSnapshot;  // used by OrgDNA export
}
```

4.2 Topology Graph

The OrgTopologyGraph is a directed graph (petgraph::DiGraph) where nodes are OrgApps, OrgExecs, and OrgCPUs, and edges are data flow relationships and event dependencies.

Node Type	Node Data	Edge Types
OrgAppNode	app_id, app_name, status, cpu_pct, last_event_at, active_record_counts	DataFlow(volume), DependsOn, SubscribesTo
OrgExecNode	exec_id, display_name, exec_type, odd_id, status, last_action_at	AssignedTo(app), ReportsTo(exec), EscalatesTo(exec)
OrgCPUNode	cpu_id, device_type, odd_id, status, last_reading_at, health_score	ConnectedTo(app), ReportsTo(exec), TriggerFor(workflow)
ExternalSystemNode	driver_id, system_name, status, last_sync_at	IntegratesWith(app), DataSource, DataSink

5. OrgDNA System Design (New in v2.0.0)

OrgDNA — Master Configuration & Replication

OrgDNA is the exportable, versionable, and replicable master configuration package of a fully configured OrgOS. It encodes everything needed to reconstruct or clone an organisation's operating system, from the kernel configuration to the chart of accounts, compliance packs, ODD templates, and RBAC policies.

OrgDNA Package Contents

Component	Contents	Purpose
Kernel Config	OrgKernel configuration: org_id, mode, enabled OrgApps, executor pool config, storage backends.	Reproduces the exact OrgOS runtime configuration
OrgApp Registry	All active OrgApps with their versions and settings. Dependency graph.	Recreates the full module configuration
ODD Library	All ODD templates and instances: employee contracts, AI agent boundaries, device configs.	Recreates the actor governance model
RBAC Policy Pack	All roles, permissions, and assignments defined for this OrgOS.	Recreates the security and access control model
Chart of Accounts	Full COA hierarchy with account codes, types, and classifications.	Recreates the financial structure
Compliance Packs	All jurisdiction-specific compliance calendars, filing obligations, and policy templates.	Recreates the compliance posture for each jurisdiction
KPI Definitions	All configured KPIs, KRI thresholds, and dashboard layouts.	Recreates the measurement and monitoring framework
Workflow Templates	All configured business processes, approval chains, and automation rules.	Recreates operational workflows
OrgOS Template Version	The version tag of the original OrgOS template (if derived from one).	Enables traceability back to the source template

OrgDNA Use Cases

Use Case	Description	Mechanism
Clone Organisation	Spin up a new OrgOS instance that is an exact operational copy of an existing one. E.g., open a new branch office.	Export OrgDNA → provision new OrgOS → import OrgDNA → adjust org_id and entity details
Franchise Deployment	Franchise owner publishes a brand OrgDNA package; franchisees clone it and operate autonomously under the brand standards embedded in the DNA.	OrgDNA Publisher in DAO module; franchisee accepts OrgDNA; smart contract enforces brand standards version compliance
Template Marketplace	Contribute a well-configured OrgOS as a reusable template in the UME Template Gallery for other organisations to use.	Sanitise OrgDNA (remove org-specific data) → submit to marketplace → publish as industry template
OrgOS Upgrade	Apply updates to OrgDNA components (new RBAC policies, updated compliance packs) across one or many OrgOS instances.	DAO OrgDNA update broadcast; member orgs receive diff notification; accept/review/merge

Simulation Baseline	Export OrgDNA as the baseline state for running simulations in the OrgSimulator.	SDO Simulator uses OrgDNA as the initial state; runs what-if scenarios

5.3 OrgDNA Rust Types

OrgDNA Struct (ume_core::orgdna)

```
#[derive(Debug, Clone, Serialize, Deserialize)]
pub struct OrgDNA {
    pub id:              OrgDNAId,
    pub org_id:          OrgId,
    pub version:         SemanticVersion,
    pub created_at:      DateTime<Utc>,
    pub published_by:    OrgExecId,
    pub parent_dna_id:   Option<OrgDNAId>,  // for cloned OrgOS instances
    pub kernel_config:   KernelConfig,
    pub app_registry:    Vec<OrgAppConfig>,
    pub odd_library:     Vec<OddTemplate>,
    pub rbac_packs:      Vec<RbacPack>,
    pub coa:             ChartOfAccounts,
    pub compliance_packs: Vec<CompliancePack>,
    pub kpi_definitions:  Vec<KpiDefinition>,
    pub workflow_templates:Vec<WorkflowTemplate>,
    pub brand_assets:    Option<BrandAssets>,
    pub dao_config:      Option<DAOConfig>,
    pub metadata:        OrgDNAMetadata,
}
```

5.4 OrgDNAManager API

API Method	Parameters	Returns	Description
export_dna()	org_id: OrgId, include_sensitive: bool	OrgDNA	Serialise current OrgOS config to OrgDNA package
import_dna()	dna: OrgDNA, target_org_id: OrgId	ImportReport	Apply OrgDNA to existing OrgOS; diff-based update
clone_dna()	source_org_id, new_org_name	OrgDNA	Produce new OrgDNA from source; new org_id assigned
diff_dna()	base: OrgDNAId, target: OrgDNAId	OrgDNADiff	Show component-level diff between two OrgDNA versions
publish_to_marketplace()	dna: OrgDNA, listing: MarketplaceListing	ListingId	Submit sanitised OrgDNA as public template
broadcast_update()	dao_id, new_dna, mandatory: bool	BroadcastResult	Push OrgDNA update to all DAO members
validate()	dna: &OrgDNA	ValidationReport	Check OrgDNA integrity: all deps satisfied, config valid

6. DAO & DLT Module Design (New in v2.0.0)

DAO & Distributed Ledger (DLT) Architecture

The UME platform enables multiple OrgOS instances to interconnect into Decentralised Autonomous Organisations (DAOs). A DAO is a network of independent but cooperating organisations, each with their own OrgOS, that operate under a shared governance model enforced by smart contracts on a distributed ledger (DLT).

DAO Architecture

Component	Description	Technical Detail
DAO Registry	Maintains the list of all member OrgOS instances, their membership status, and their consensus node addresses.	Stored in DAO OrgApp; replicated to DLT genesis block
DLT Layer	The shared distributed ledger. All inter-organisation transactions, smart contract events, OrgDNA updates, and governance votes are recorded as immutable blocks.	Pluggable consensus: default PBFT for permissioned DAOs; configurable for public chain anchoring
Smart Contract Engine	Executes predetermined agreements automatically when trigger conditions are met. Built on the DLT layer.	Smart contracts defined in UME visual builder; compiled to bytecode; executed by SC Engine on DLT nodes
OrgDNA Broadcast	The parent org (or designated governance admin) can publish OrgDNA updates that all member OrgOS instances receive and can accept or review.	Event-driven push via DAO event bus; versioned diffs; multi-sig acceptance for mandatory updates
Inter-Org Event Bus	Extends the OrgKernel event bus across organisation boundaries, enabling cross-org event subscriptions and notifications.	Federated NATS or Kafka cluster; org-to-org topic namespacing; mTLS between OrgOS nodes
Governance Engine	Implements the voting model defined in the DAO governance smart contract. Manages proposals, voting periods, quorum checks, and execution of approved decisions.	On-chain voting state; off-chain proposal authoring; automatic execution on quorum reached
Treasury Manager	Manages the DAO shared treasury (if configured): balances, multi-sig withdrawals, investment policies, fund allocation rules.	Multi-sig wallet; automatic allocation rules from smart contracts; full DLT transaction history

DLT Transaction Types

Transaction Type	Trigger	Participants	Effect
ORGOS_JOIN	New org joins DAO	Joining org + DAO governance	Member added; OrgDNA snapshot stored; voting weight assigned
ROYALTY_PAYMENT	Smart contract trigger on revenue event	Franchisee org → Parent org	Automatic treasury transfer; royalty record created; DLT block committed
ORGDNA_UPDATE	Parent publishes OrgDNA update	Parent org → all members	Update broadcast; member orgs notified; acceptance tracking on-chain

GOVERNANCE_VOTE	Governance proposal submitted	All member orgs	Vote recorded; quorum tracked; automatic execution on threshold met
SMART_CONTRACT_EXEC	Any contract trigger condition met	Counterparty orgs	Contract executed; state updated; funds/resources transferred; DLT block committed
BRAND_STANDARDS_CHECK	Periodic compliance evaluation	DAO governance → member org	OrgDNA version validated; breach detected if out of compliance; cure period started
RESOURCE_TRANSFER	Inter-org resource or data exchange	Source org → destination org	Asset/data transferred; ownership recorded on DLT; both orgs' SDOs updated
ORGOS_LEAVE	Org exits DAO	Leaving org + DAO governance	Exit terms executed per smart contract; outstanding obligations settled; membership removed

6.3 DLT Node Design

DLT Node (ume_modules::dao::dlt)

```
pub struct DLTNode {
    node_id:        NodeId,
    org_id:         OrgId,
    consensus:      PBFTConsensus,
    block_store:    Arc<BlockStore>,
    mempool:        Arc<Mempool>,
    sc_engine:      SmartContractEngine,
    peers:          Vec<PeerConnection>,
}

impl DLTNode {
    pub fn submit_tx(&self, tx: DLTTransaction) -> UmeResult<TxHash>;
    pub fn get_block(&self, n: u64) -> Option<DLTBlock>;
    pub fn query_ledger(&self, filter: LedgerFilter) -> Vec<DLTBlock>;
    pub fn verify_tx(&self, tx_hash: &TxHash) -> VerificationResult;
    pub fn deploy_contract(&self, contract: SmartContract) -> ContractId;
}
```

6.4 Smart Contract Execution

Contract Term Type	Rust Type	Execution Trigger	Effect
RoyaltyPayment	RoyaltyTerm{rate_pct, source_app, payment_app}	OrgApp Finance.period_closed event	Auto-debit royalty_pct of period revenue; credit parent treasury; commit ROYALTY_PAYMENT block
BrandStandardsCheck	BrandStandardsTerm{max_dna_version_lag}	Periodic (configurable interval)	Compare member OrgDNA version to parent; log BRAND_STANDARDS_CHECK; trigger cure period on breach

GovernanceVote	GovernanceTerm{quorum, threshold, voting_period_days}	Proposal submitted to DAO governance	Record votes on DLT; auto-execute approved action on quorum reached
ResourceTransfer	ResourceTransferTerm{resource_type, trigger_app, trigger_event}	Any configured OrgApp event	Transfer specified resource between org treasuries; record RESOURCE_TRANSFER block
SLAEnforcement	SLATerm{metric, threshold, penalty_fn}	Periodic metric evaluation	Evaluate SLA metric from specified OrgApp; apply penalty function on breach; log SLA_BREACH block

7. Updated Module Designs (v2.0.0)

Existing module designs from v1.0.0 remain in full force. The following describes the v2.0.0 additions to key modules.

7.1 Finance Module — v2.0.0 Additions

Addition	Description	Technical Detail
OrgDNA Component	Finance exports COA, account configuration, period definitions, and tax settings to OrgDNA	to_orgdna_component() serialises FinanceConfig + ChartOfAccounts to OrgDNAComponent
DLT Integration	Financial events (period close, invoice settlement) can trigger DLT transactions for DAO royalty smart contracts	Finance module publishes finance.period.closed event; DAOClient subscribes and triggers smart contract evaluation
OrgExec Action Validation	All financial postings validated against posting OrgExec's ODD (data access grants, approval thresholds)	KernelFacade calls ODDBus.validate_action() before accepting journal entry; ODD specifies max posting value, approval requirements

7.2 HR Module — v2.0.0 Additions

Addition	Description	Technical Detail
OrgExec Onboarding	New hire wizard generates an ODD contract for the new OrgExec based on role and employment type template	HRModule::create_employee() calls ODDBus::register() with generated ODD; ODD stored as OrgExec record
ODD Template Management	HR module manages ODD templates for all employee roles and contractor types	ODD templates stored in OrgDNA; HR module exposes ODD template CRUD via kernel facade
AI Agent OrgExec Support	HR module supports registering AI agents as OrgExecs with AI-AGENT-ODD template	OrgExecType::AIAgent path in new hire wizard; special ODD capabilities/constraints for AI
OrgDNA Component	HR exports role definitions, ODD templates, and org structure to OrgDNA	to_orgdna_component() includes OrgStructure + OddTemplates + RoleDefinitions

7.3 New: OrgExec & ODD Manager OrgApp (App #43)

A new dedicated OrgApp provides the UI and API surface for managing all OrgExecs and their ODD contracts.

Feature	Description
OrgExec Registry	List all OrgExecs (humans, AI agents, OrgCPUs) with type, ODD version, status, last active
ODD Editor	Visual ODD contract editor: capabilities checklist, constraint configuration, data access grant builder, escalation rule setup
ODD Version History	Full version history of each ODD contract; diff viewer; rollback capability

Action Audit Log	Per-OrgExec audit log of all actions, ODD decisions (Allow/Deny/Escalate), and violations
OrgCPU Dashboard	IoT device and machine status panel; OrgCPU health monitoring; maintenance alert integration
AI Agent Console	Dedicated view for AI agent OrgExecs: task history, confidence scores, escalation frequency, cost tracking

7.4 New: DAO & DLT OrgApp (App #44)

Feature	Description
DAO Overview Dashboard	Member network graph, treasury balances, recent ledger activity, pending governance votes
Smart Contract Builder	Visual no-code contract term editor; generates compiled smart contract from term definitions
OrgDNA Publisher	Publish, version, and broadcast OrgDNA updates to DAO members; track acceptance status
DLT Ledger Viewer	Browse all DLT blocks with hash verification; filter by tx type, org, and time range
Governance Portal	Submit proposals, vote on governance decisions, view results; quorum tracking; auto-execution on threshold
Member Management	Invite/remove DAO members; manage membership status; view per-member OrgDNA compliance

Appendix E: Updated Event Catalogue (v2.0.0 Additions)

Event Topic	Payload	Emitted By	Subscribers
odd.action.allowed	exec_id, action, odd_id, timestamp	ODDBus	Audit log, SDO engine
odd.action.denied	exec_id, action, odd_id, reason, timestamp	ODDBus	Audit log, SDO engine, Security module
odd.action.escalated	exec_id, action, odd_id, rule_id, escalate_to	ODDBus	Notification service, HR module, Work module
odd.violation.recorded	exec_id, constraint_id, action, timestamp	ODDBus	Audit log, Risk module
odd.contract.updated	exec_id, old_odd_version, new_odd_version	ODD Manager App	Audit log, SDO engine
sdo.topology.updated	org_id, node_id, change_type, timestamp	SDO Engine	Portal/Dashboard module
sdo.state.updated	org_id, app_id, state_delta	SDO Engine	Portal/Dashboard, Analytics module
orgdna.version.created	org_id, version, author	OrgDNA Manager	Audit log, DAO client
orgdna.broadcast.sent	dao_id, version, mandatory, recipient_count	DAO Module	Audit log
dao.member.joined	dao_id, org_id, block_num	DLT Node	Audit log, DAO dashboard
dao.tx.committed	dao_id, tx_type, tx_hash, from_org, to_org	DLT Node	Audit log, Finance module (for royalty)
dao.vote.completed	dao_id, proposal_id, result, quorum_reached	Governance Engine	Audit log, all DAO members
dao.contract.executed	dao_id, contract_id, tx_hash, effect_summary	Smart Contract Engine	Audit log, Finance module
dao.brand_standards.breach	dao_id, member_org_id, dna_version_lag	Smart Contract Engine	DAO governance, member HR module
orgcpu.status.updated	cpu_id, old_status, new_status, health_score	OrgCPU Registry	SDO engine, Operations module, Risk module
orgsim.result.ready	org_id, scenario_id, result_summary	OrgSimulator	Dashboard module, requesting OrgExec

Appendix F: Updated Glossary

New Concepts & Terminology (v2.0.0)

The following new concepts were introduced in v2.0.0 of the UME platform. All prior documents that reference "UME platform", "kernel", "modules", and "device drivers" should be read in conjunction with these updated definitions.

Term	Symbol	Definition
OrgOS	OrgOS	A configured, running Organisation Operating System produced by the UME platform. Every organisation provisioned on UME receives a dedicated OrgOS instance, which is the combination of an OrgKernel and its set of active OrgApps.
OrgOS Factory	UME Platform	The UME platform itself is the factory that produces, configures, deploys, and maintains OrgOS instances. UME is a meta-operating-system: an operating system that creates and manages operating systems.
OrgKernel	OrgKernel	The running instance of the UME kernel for a specific organisation. The OrgKernel is the composition root, resource manager, and supervisor for all OrgApps within a single organisation's OrgOS.
OrgApp	OrgApp	A modular application running within an OrgOS, implementing a specific organisational capability (Finance, HR, Legal, etc.). Previously called "organisation modules". OrgApps are the user-facing applications that manage and run the organisation.
OrgExec	OrgExec	An executor of work within an OrgOS: a human employee, AI agent, hybrid human-AI system, or any autonomous actor that receives tasks and executes them on behalf of the organisation. OrgExecs are governed by ODD contracts.
OrgCPU	OrgCPU	A digital-only executor within an OrgOS: machines, IoT devices, sensors, 3rd-party platforms, manufacturing equipment, POS terminals, and any non-human device that executes work. OrgCPUs are governed by ODD device driver contracts.
ODD	OrgDeviceDriver (ODD)	A formal contract that defines how a specific OrgExec or OrgCPU operates within the OrgOS. Specifies capabilities (what it can do), constraints (what it must not do), data access rights, rate limits, escalation triggers, and audit requirements. ODDs are the primary mechanism for governing all actors in an OrgOS.
OrgDNA	OrgDNA	The exportable, versionable master configuration package of a fully configured OrgOS. OrgDNA captures the OrgKernel config, all active OrgApps and their settings, ODD templates, RBAC policies, chart of accounts, KPIs, and jurisdiction compliance packs. OrgDNA can be cloned, forked, shared with franchisees, or published as a template.
SDO	Software Defined Organisation (SDO)	The digital twin of a physical organisation. An SDO is the complete runtime representation of an organisation within the UME platform — all running OrgApps, all OrgExecs, all OrgCPUs, all data flows, and all organisational state. Users operate at the SDO layer; the physical organisation is realised through the digital twin.

OrgRuntime	Organisation Runtime	The live execution of an OrgOS: the combination of the OrgKernel, all running OrgApps, all active OrgExecs, all connected OrgCPUs, and all live organisational data. The OrgRuntime realises organisation outcomes in real time.
OrgMemory	OrgMemory / DLT	The persistent, immutable record of all significant organisational events and inter-organisation transactions. Within a single OrgOS, OrgMemory is maintained by the OrgKernel audit chain. Across multiple OrgOS instances (DAO), OrgMemory is a Distributed Ledger (DLT) shared among all member organisations.
DAO	Decentralised Autonomous Organisation (DAO)	A set of interconnected, independent OrgOS instances that cooperate under a shared governance model. DAOs are formed through the UME DAO module, which provides smart contract governance, distributed ledger infrastructure, shared OrgDNA versioning, and inter-organisation event routing.
DLT	Distributed Ledger Technology (DLT)	The inter-organisation ledger that records all cross-OrgOS transactions, smart contract executions, OrgDNA updates, and governance votes for a DAO. The DLT is the shared OrgMemory of a DAO — immutable, cryptographically verified, and replicated across all member OrgOS nodes.
Smart Contract	Smart Contract	An automatically executable agreement between two or more OrgOS instances within a DAO, stored on the DLT. Smart contracts govern resource transfers (royalties, payments), brand standards enforcement, governance decisions, and any other inter-organisation obligation.
OrgExec Type	OrgExec Classification	Human (natural persons employed/contracted), AI Agent (LLM-powered or rule-based autonomous agent), Hybrid (human-AI collaboration system), or Physical Agent (OrgCPU: device/machine/sensor).